



Universidade Estadual Paulista “Júlio de Mesquita Filho” (UNESP)
Faculdade de Ciências – Bauru
Departamento de Computação

RANK-IA

Formação otimizada de equipes com Inteligência Artificial

Engenharia de Software II

Fernando Hiroshi Murusaki – RA: 241025851

Igor dos Reis Gomes – RA: 241025265

Matheus Santos Magro – RA: 231025335

Murilo Tomaz Gonzaga – RA: 241024684

Sumário

Como utilizar este modelo	1
1 Introdução e Objetivos	2
1.1 Contexto do software	2
1.2 Escopo	3
1.3 Visão geral dos requisitos	4
1.4 Objetivos de qualidade	5
2 Arquitetura do Sistema	7
2.1 Contexto e fronteiras do sistema	7
2.2 Estilo ou padrão arquitetural	7
2.3 Contêineres e componentes	7
2.4 Decisões arquiteturais relevantes	7
2.5 Restrições de arquitetura	8
3 Projeto de Componentes	9
3.1 Visão geral dos componentes	9
3.2 Detalhamento dos componentes principais	9
3.3 Princípios de projeto	9
3.4 Padrões de projeto	10
4 Projeto de Interface do Usuário	11
5 Especificação e Estratégia de Testes	12
5.1 Planejamento e níveis de teste	12
5.2 Testes de unidade	12
5.3 Testes de integração	12
5.4 Critérios de conclusão e cobertura	12
5.5 Automação e ferramentas	12
6 Gestão de Configuração e Manutenção	13
6.1 Repositório e itens de configuração	13
6.2 Gestão de dependências e alterações	13
6.3 Rastreabilidade	13
6.4 <i>Build</i> , integração e entrega	13

Como utilizar este modelo

Este documento é um roteiro para apoiar a documentação do projeto final. Os textos marcados como *Orientação* explicam o que se espera em cada parte e podem ser ocultados alterando-se, no início do arquivo, `\orientacoestrue` para `\orientacoesfalse`.

O documento deve registrar as principais decisões do projeto, e não apenas reunir diagramas. Sempre que uma escolha relevante for apresentada, expliquem brevemente a motivação e suas consequências. Diagramas, tabelas e protótipos devem ser mencionados e explicados no texto.

Não é necessário documentar todas as classes, telas ou detalhes internos do sistema. Em projetos maiores, priorizem os elementos mais relevantes, complexos ou representativos.

Capítulo 1

Introdução e Objetivos

***Orientação:** Apresentem brevemente o sistema e expliquem o que será documentado neste capítulo. O objetivo é permitir que um leitor compreenda o problema e o escopo antes de examinar as decisões de projeto.*

O RANK-IA é uma aplicação web corporativa que apoia a formação de equipes de trabalho dentro de uma organização. A partir das habilidades, competências e lacunas de formação (*gaps*) registradas para cada colaborador, o sistema utiliza um modelo de Inteligência Artificial para sugerir composições de equipe e associa a cada sugestão um **Coefficiente de Aproveitamento**, que estima o desempenho esperado do grupo. Como a formação de equipes envolve fatores humanos que nem sempre são mensuráveis, as sugestões podem ser ajustadas manualmente pelo gestor, e esses ajustes são registrados e devolvidos ao modelo como retroalimentação. O produto é, portanto, deliberadamente híbrido: automação assistida por supervisão humana, e não decisão automática.

Este capítulo delimita o problema tratado, o escopo do sistema, os requisitos mais relevantes para compreendê-lo e os atributos de qualidade que orientam as decisões de projeto apresentadas nos capítulos seguintes. A especificação detalhada de requisitos, os casos de uso e os diagramas de análise foram produzidos na etapa anterior do projeto e são aqui referenciados de forma resumida, sem repetição integral.

1.1 Contexto do software

***Orientação:** Contextualizem o problema do ponto de vista dos usuários e da atividade que o software deve apoiar, automatizar ou melhorar. Identifiquem os principais usuários ou partes interessadas quando isso for relevante.*

Na maioria das organizações, a montagem de equipes para um projeto ou tarefa é uma atividade manual e pouco sistematizada. O profissional de Recursos Humanos ou o gestor responsável precisa cruzar currículos, planilhas de competências, histórico de projetos anteriores e conhecimento tácito sobre o perfil de cada pessoa. Esse processo apresenta três limitações recorrentes:

- **Custo de tempo.** A informação necessária está dispersa em documentos de formatos distintos, e o esforço de consolidá-la cresce rapidamente com o número de colaboradores considerados.
- **Subjetividade.** Sem um critério explícito, a escolha depende fortemente da familiaridade do gestor com os candidatos, o que tende a favorecer sempre os mesmos perfis e a deixar competências pouco visíveis fora das composições.
- **Ausência de rastreabilidade.** Não fica registrado por que uma determinada equipe foi formada, o que dificulta revisar a decisão, comparar alternativas e aprender com os resultados obtidos.

O RANK-IA atua exatamente sobre essa atividade. Ele centraliza os dados de competências dos colaboradores, torna explícito o critério de composição por meio do Coeficiente de Aproveitamento, e mantém o histórico das equipes geradas e dos ajustes manuais realizados. Um efeito secundário relevante é que as lacunas identificadas durante a análise passam a alimentar sugestões de treinamento, ligando a formação de equipes ao desenvolvimento profissional dos colaboradores.

As principais partes interessadas do sistema estão descritas na Tabela 1.1.

Tabela 1.1: Partes interessadas do RANK-IA.

Parte interessada	Tipo	Interesse no sistema
Gestor / RH	Usuário primário	Cadastrar colaboradores, parametrizar os critérios da IA, solicitar e validar equipes, acompanhar relatórios. Possui acesso total ao sistema.
Colaborador	Usuário secundário	Consultar as próprias competências e <i>gaps</i> , acompanhar treinamentos sugeridos e visualizar as equipes das quais participa. Possui acesso restrito aos seus próprios dados.
Organização	Patrocinador	Reduzir o tempo de montagem de equipes, aumentar a produtividade dos times e manter a operação em conformidade com a LGPD.
Equipe de desenvolvimento	Executor	Evoluir o modelo de IA e os módulos do sistema sem regressões, com apoio de testes automatizados.

1.2 Escopo

Orientação: *Delimitem o que faz parte do sistema e, quando necessário, o que está explicitamente fora do escopo. Evitem transformar esta seção em uma especificação detalhada de requisitos.*

O escopo do RANK-IA compreende o ciclo completo da formação de uma equipe, desde o registro das competências dos colaboradores até o acompanhamento dos resultados da equipe formada. Fazem parte do sistema:

- **Gerenciamento de colaboradores** — cadastro manual por formulário ou importação assistida a partir de currículos em PDF, com extração automática de habilidades e experiências.
- **Geração de equipes por IA** — composição automática a partir dos critérios informados pelo gestor (tamanho da equipe, habilidades exigidas, peso da experiência), com cálculo do Coeficiente de Aproveitamento para cada sugestão.
- **Ajuste manual com retroalimentação** — interface para substituir, incluir ou remover membros de uma sugestão; cada alteração é registrada e utilizada como *feedback* para o modelo.
- **Relatórios e indicadores** — visualização gráfica da qualidade das equipes e relatórios de desempenho individual e coletivo.
- **Sugestão de treinamentos** — recomendação de capacitações associadas às lacunas identificadas no perfil dos colaboradores.

- **Controle de acesso por perfil** — autenticação e autorização diferenciadas para os perfis Gestor/RH e Colaborador, com tratamento dos dados pessoais conforme a LGPD. Estão explicitamente **fora do escopo** deste projeto:
- Processos de RH não relacionados à composição de equipes, como folha de pagamento, controle de ponto, recrutamento externo e avaliação de desempenho formal.
- Gestão de projetos e de tarefas: o sistema forma a equipe, mas não acompanha a execução do trabalho que ela realiza.
- Integração com sistemas de RH ou ERP de terceiros, prevista apenas por meio da exportação de relatórios em formatos abertos.
- Aplicativo móvel nativo; o acesso se dá por navegador, com interface responsiva.
- Qualquer decisão automática de contratação, promoção ou desligamento. O sistema é uma ferramenta de apoio à decisão, e a validação humana é sempre obrigatória.

1.3 Visão geral dos requisitos

***Orientação:** Apresentem os requisitos funcionais e não funcionais mais importantes para compreender o projeto. Caso a equipe tenha uma especificação detalhada de requisitos produzida anteriormente, ela pode ser incluída em um apêndice ou referenciada neste documento.*

A especificação detalhada de requisitos, incluindo regras de negócio, casos de uso e histórias de usuário, foi elaborada na disciplina anterior e permanece válida como documento de referência. Esta seção reúne apenas os requisitos que influenciam diretamente as decisões de projeto discutidas neste documento.

A Tabela 1.2 apresenta os requisitos funcionais essenciais.

Tabela 1.2: Requisitos funcionais essenciais.

ID	Descrição
RF01	O sistema deve coletar e armazenar informações relevantes sobre os colaboradores, como habilidades, experiências, histórico de desempenho e lacunas de formação.
RF02	O sistema deve gerar automaticamente equipes balanceadas a partir dos atributos dos colaboradores e dos critérios definidos pelo gestor.
RF03	O sistema deve calcular e exibir o Coeficiente de Aproveitamento de cada equipe sugerida, permitindo a comparação entre composições alternativas.
RF04	O sistema deve permitir a troca manual de membros nas equipes sugeridas e registrar cada ajuste como <i>feedback</i> para o modelo de IA.
RF05	O sistema deve identificar as lacunas de cada colaborador e sugerir treinamentos correspondentes.
RF06	O sistema deve gerar relatórios de desempenho individual e coletivo e apresentar a qualidade das equipes de forma gráfica.
RF07	O sistema deve restringir o acesso às funcionalidades conforme o perfil do usuário (Gestor/RH ou Colaborador).

Os requisitos não funcionais mais determinantes para a arquitetura estão resumidos na Tabela 1.3, acompanhados da métrica adotada para verificá-los.

Vale destacar o RNF07: ele não decorre de uma exigência do cliente, mas da natureza experimental do componente de IA. Como o modelo será ajustado repetidamente ao longo da vida do sistema, tratá-lo como uma dependência substituível é uma decisão que atravessa toda a arquitetura descrita no Capítulo 2.

Tabela 1.3: Requisitos não funcionais determinantes para o projeto.

ID	Descrição	Métrica de verificação
RNF01	As senhas dos usuários devem ser armazenadas de forma criptografada.	Uso de algoritmo de <i>hash</i> forte (<i>bcrypt</i>); nenhuma senha em texto claro na base de dados.
RNF02	O tratamento de dados pessoais deve seguir a LGPD.	Ausência de vulnerabilidades de severidade alta ou crítica em teste de intrusão; acesso aos dados restrito por perfil.
RNF03	O sistema deve responder à solicitação de formação de equipe em tempo adequado.	Mais de 90% das equipes formadas dentro do tempo esperado, independentemente do tamanho do time.
RNF04	O sistema deve suportar aumento de carga de usuários simultâneos.	Tempo médio de resposta ≤ 2 s e taxa de erro $\leq 1\%$ em teste de carga com 50.000 conexões simultâneas.
RNF05	A interface deve ser utilizável por gestores sem treinamento especializado.	Pontuação superior a 70 na escala SUS e ajuste manual de equipe realizável em até três cliques.
RNF06	O sistema deve funcionar nos principais navegadores e resoluções de tela.	Renderização correta em Chrome, Firefox e Edge, e em resoluções a partir de 1024 px, sem instalação de <i>plugins</i> .
RNF07	O modelo de IA deve poder ser retreinado e substituído sem alteração da lógica de negócio.	Troca da implementação do serviço de IA sem modificar as classes de negócio, validada por testes com <i>mocks</i> .

1.4 Objetivos de qualidade

Orientação: *Identifiquem os atributos de qualidade que realmente influenciam as decisões de projeto, por exemplo: manutenibilidade, desempenho, segurança, confiabilidade, usabilidade ou portabilidade. Podem ser utilizados modelos como ISO/IEC 25010 ou FURPS+, desde que apenas os atributos relevantes ao sistema sejam selecionados.*

Os objetivos de qualidade do RANK-IA foram definidos a partir do modelo de qualidade de produto da norma ISO/IEC 25010 [International Organization for Standardization, 2011], cujas características estão ilustradas na Figura 1.1. Das oito características previstas pela norma, foram selecionadas apenas aquelas que efetivamente restringem as decisões de projeto; as demais foram consideradas satisfeitas pelas práticas usuais de desenvolvimento web e não são discutidas neste documento.

A Tabela 1.4 sintetiza os objetivos priorizados e o que cada um implica concretamente no projeto do sistema.

Os dois primeiros objetivos merecem uma observação, porque tensionam entre si. A adequação funcional pressiona a equipe a experimentar com o modelo de IA, o que tende a gerar código acoplado ao algoritmo do momento; a manutenibilidade exige o contrário. A escolha do projeto foi privilegiar a manutenibilidade na fronteira entre o domínio e a IA — isolando o modelo atrás de uma abstração — de modo que a experimentação fique contida em um único componente substituível. As consequências dessa decisão são detalhadas no Capítulo 2 e no Capítulo 3.

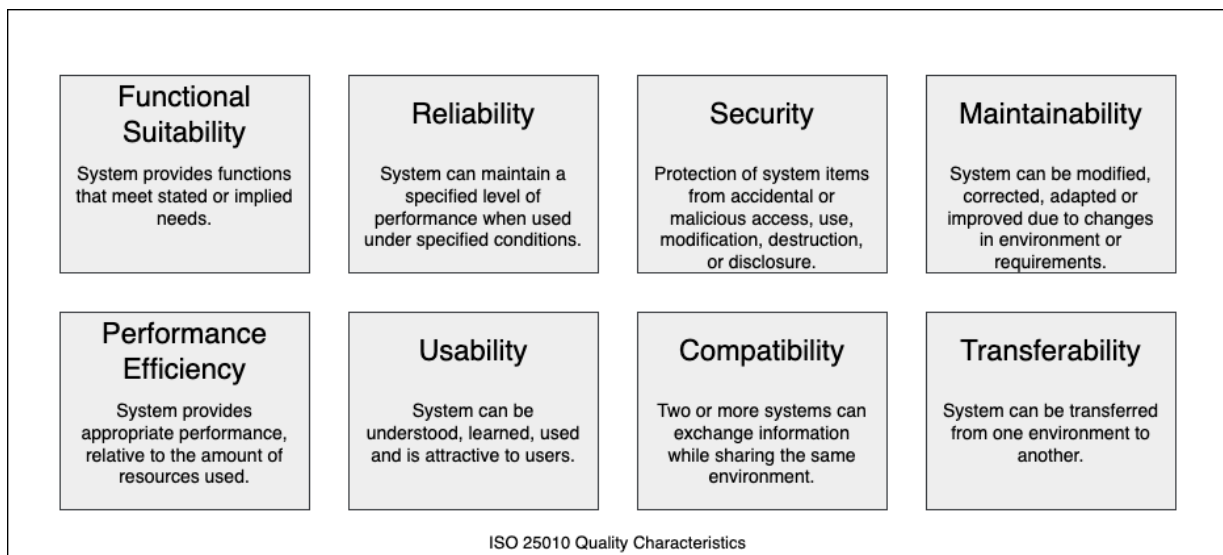


Figura 1.1: Características do modelo de qualidade de produto da ISO/IEC 25010.

Tabela 1.4: Objetivos de qualidade do RANK-IA.

Objetivo	Prioridade	Impacto esperado no projeto
Manutenibilidade	Alta	Modularização por coesão em torno do domínio de equipes, dependências expressas por interfaces (inversão de dependência) e testes automatizados. É o que torna viável substituir o modelo de IA sem reescrever a lógica de negócio.
Adequação funcional	Alta	A qualidade da sugestão é o valor central do produto: o Coeficiente de Aproveitamento precisa ser calculado de forma correta e explicável, e a sugestão deve ser aproveitável sem que o gestor a descarte por completo.
Segurança	Alta	Autenticação, autorização por perfil, criptografia de senhas e das comunicações, e segregação dos dados pessoais dos colaboradores em conformidade com a LGPD.
Confiabilidade	Alta	Persistência garantida das equipes salvas e dos ajustes manuais, com versionamento das composições para auditoria; para uma mesma entrada e um mesmo modelo, a sugestão deve ser reproduzível.
Usabilidade	Média	Interface consistente, ajuste manual em poucas etapas e apresentação gráfica dos indicadores, de modo que o gestor compreenda a composição sugerida sem conhecimento técnico sobre o modelo.
Eficiência de desempenho	Média	Separação do processamento da IA em serviço próprio, evitando que o custo da inferência degrade as demais operações da aplicação.
Portabilidade e compatibilidade	Baixa	Aplicação web responsiva, sem dependência de <i>plugins</i> , e leitura e exportação em formatos abertos (PDF, CSV).

Capítulo 2

Arquitetura do Sistema

Orientação: Apresentem a arquitetura proposta e justifiquem as principais decisões. O leitor deve conseguir compreender a estrutura geral do sistema antes de entrar nos detalhes de seus componentes.

2.1 Contexto e fronteiras do sistema

Orientação: Apresentem um diagrama de contexto mostrando o sistema, seus principais usuários e os sistemas externos com os quais ele se comunica. Expliquem as relações mostradas no diagrama.

2.2 Estilo ou padrão arquitetural

Orientação: Identifiquem o(s) estilo(s) ou padrão(ões) arquitetural(is) adotado(s), quando aplicável, e justifiquem a escolha considerando as características do sistema e seus objetivos de qualidade.

2.3 Contêineres e componentes

Orientação: Apresentem uma visão de alto nível da decomposição do sistema. Pode-se utilizar, quando adequado, os níveis de contexto, contêiner e componente do modelo C4. Não basta inserir os diagramas: descrevam a responsabilidade dos elementos relevantes e as principais dependências ou comunicações entre eles.

2.4 Decisões arquiteturais relevantes

Orientação: Registrem as decisões que tiveram impacto significativo na arquitetura. Para cada decisão, apresentem de forma breve a motivação, alternativas consideradas quando pertinente e principais consequências.

Tabela 2.1: Modelo para registrar decisões arquiteturais.

Decisão	Motivação	Consequências
Exemplo	Por que essa alternativa foi escolhida?	Benefícios, limitações ou compromissos envolvidos.

2.5 Restrições de arquitetura

Orientação: Registrem restrições que limitem as decisões de arquitetura ou implementação, como tecnologias obrigatórias, integrações com sistemas existentes, ambiente de execução, restrições organizacionais ou convenções. Incluam apenas restrições efetivamente relevantes ao projeto.

Capítulo 3

Projeto de Componentes

Orientação: Detalhem a organização interna das partes mais relevantes do sistema. O projeto de componentes pode ser representado graficamente, textualmente ou por tabelas, de acordo com a natureza da solução.

3.1 Visão geral dos componentes

Orientação: Listem os principais componentes e suas responsabilidades. Para sistemas maiores, esta visão geral pode cobrir todos os componentes, enquanto o detalhamento das seções seguintes pode se concentrar apenas nos mais relevantes ou complexos.

Tabela 3.1: Principais componentes do sistema.

Componente	Responsabilidade principal
Componente A	Descrever sua responsabilidade principal.
Componente B	Descrever sua responsabilidade principal.

3.2 Detalhamento dos componentes principais

Orientação: Seleccionem os componentes mais relevantes ou complexos e apresentem sua estrutura, responsabilidades, interfaces e dependências. Se o projeto for orientado a objetos, utilizem principalmente diagramas de classes UML e, quando necessário, complementem com diagramas de sequência, atividade ou outros artefatos.

3.3 Princípios de projeto

Orientação: Expliquem como os princípios de projeto discutidos na disciplina foram aplicados nos principais componentes ou classes do sistema. Por exemplo, podem ser discutidos aspectos relacionados à separação de responsabilidades, coesão, acoplamento, dependência de abstrações e princípios SOLID. Não apenas enumerem os princípios: indiquem onde foram aplicados e qual problema de projeto ajudaram a resolver. Para evidenciar sua aplicação, utilizem, quando adequado, diagramas de classes ou trechos de código-fonte.

3.4 Padrões de projeto

Orientação: Para cada padrão de projeto efetivamente utilizado, indiquem o problema que motivou sua aplicação, as classes ou componentes participantes, como o padrão foi aplicado e qual benefício se espera obter. Não utilizem padrões apenas para preencher esta seção.

Capítulo 4

Projeto de Interface do Usuário

Orientação: *Apresentem as interfaces e os principais fluxos de interação do usuário com o sistema. O principal artefato pode ser constituído por protótipos de baixa ou alta fidelidade. Diagramas de atividade, fluxos de navegação ou outros artefatos podem complementar a explicação.*

Não é necessário apresentar todas as telas. Priorizem as interfaces e os fluxos mais importantes para compreender o funcionamento do sistema.

Orientação: *Para cada fluxo principal, expliquem brevemente a tarefa do usuário e as decisões de interface relevantes. Se a equipe utilizar uma ferramenta externa, como Figma, apresentem exemplos representativos neste documento e forneçam o link para o material completo.*

Capítulo 5

Especificação e Estratégia de Testes

Orientação: Mostrem como a equipe pretende verificar o comportamento do software e reduzir o risco de defeitos. O capítulo deve apresentar uma estratégia coerente, e não apenas uma lista de ferramentas.

5.1 Planejamento e níveis de teste

Orientação: Indiquem os níveis de teste aplicáveis ao projeto, por exemplo, unidade, integração e sistema, e expliquem o objetivo de cada um deles no contexto do software.

5.2 Testes de unidade

Orientação: Seleccionem unidades relevantes e definam cenários e casos de teste. Quando existirem dependências externas ou difíceis de controlar, expliquem se serão utilizados stubs, mocks ou outras formas de substituição. Apresentem exemplos representativos contendo entradas, condições e resultados esperados.

5.3 Testes de integração

Orientação: Expliquem como os componentes serão integrados e testados. Quando fizer sentido, indiquem a estratégia adotada (por exemplo, top-down, bottom-up, depth-first ou breadth-first) e justifiquem-na. Descrevam também como testes de regressão serão utilizados após alterações.

5.4 Critérios de conclusão e cobertura

Orientação: Definam critérios que permitam decidir quando a atividade de teste atingiu um nível satisfatório. A cobertura de código pode ser um dos indicadores, mas não deve ser tratada isoladamente como garantia de qualidade. Se uma meta quantitativa for adotada, indiquem o tipo de cobertura e justifiquem a meta.

5.5 Automação e ferramentas

Orientação: Descrevam as ferramentas utilizadas ou planejadas para executar testes, gerar relatórios e, quando aplicável, automatizar sua execução.

Capítulo 6

Gestão de Configuração e Manutenção

***Orientação:** Este capítulo deve mostrar como os artefatos do software são versionados, modificados e mantidos de forma controlada. Relacionem as práticas descritas ao projeto da equipe, evitando apenas reproduzir definições teóricas.*

6.1 Repositório e itens de configuração

***Orientação:** Informem o repositório utilizado e identifiquem os principais itens de configuração que precisam ser controlados, por exemplo: código-fonte, arquivos de configuração, scripts, documentação, modelos de dados e testes.*

6.2 Gestão de dependências e alterações

***Orientação:** Expliquem como dependências externas são registradas e versionadas e como alterações relevantes são controladas. Quando uma alteração afetar vários artefatos, indiquem como a equipe mantém a coerência entre código, testes, documentação e demais elementos relacionados.*

6.3 Rastreabilidade

***Orientação:** Quando aplicável, mostrem como uma alteração pode ser rastreada entre os artefatos relacionados, por exemplo: necessidade/requisito, tarefa ou issue, componente afetado, commit e caso de teste. Não é necessário criar uma matriz extensa; um ou dois exemplos relevantes podem ser suficientes para demonstrar a prática.*

6.4 *Build*, integração e entrega

***Orientação:** Descrevam como o software é construído e executado. Se houver automação de build, testes, integração contínua ou entrega/deploy, apresentem o fluxo adotado e as ferramentas envolvidas. Caso essas práticas não sejam utilizadas, expliquem brevemente como o processo é realizado manualmente.*

Controle de Versões do Documento

Orientação: Registrem apenas alterações significativas nesta documentação. Este controle se refere ao documento; o versionamento do software deve ser descrito no capítulo de Gestão de Configuração e Manutenção.

Tabela 6.1: Histórico de versões do documento.

Data	Autor(es)	Versão	Alteração
DD/MM/AAAA	Equipe	1.0	Versão inicial para entrega.

Referências bibliográficas

International Organization for Standardization. *ISO/IEC 25010:2011 – Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models*. ISO, 2011.